TECHNICAL SPECIFICATION & IMPLEMENTATION GUIDE

DNS Architecture Implementation Guide (with Technical Rationale)

Pre-Implementation Checklist

Requirement	Value / Target (NAT Mode - Recommended)	Value / Target (Bridged Mode)
Physical Host OS	Windows 10 / 11 with VMware Workstation	Windows 10 / 11 with VMware Workstation
Guest Virtual Machine	Windows Server (2019 / 2022 / 2025)	Windows Server (2019 / 2022 / 2025)
VMware Network Mode	**NAT (VMnet8)** [Works anywhere, immune to Wi-Fi changes]	**Bridged (VMnet0)** [Direct L2 connection to physical LAN]
Primary Static IP for Server	**`192.168.1.10`** (Windows DNS & Active Directory)	**`192.168.100.50`** (or matching router subnet)
Secondary Static IP for AdGuard	**`192.168.1.11`** (AdGuard Home Port 53)	**`192.168.100.51`**
Default Gateway	**`192.168.1.1`** (VMware Virtual NAT Gateway)	**`192.168.100.1`** (Physical Home Router)
Client Devices	Virtual Machine clients (`pro-win-client`, `pro-win-client2`)	Physical hardware (Smartphones, Smart TVs, external PCs)
Docker Engine on Server	Docker Desktop with WSL2 backend	Docker Desktop with WSL2 backend

📌 Master Subnet IP Allocation Schema (`192.168.1.0/24`)

Use this IP plan to ensure other VMs, containers, and servers do not conflict:

IP Address Range	Assigned Role / Machine	Status	Can Other Servers Use This IP?
`192.168.1.1`	VMware Virtual NAT Gateway	🔒 Active	❌ **NO** (Network Gateway)
`192.168.1.10`	Windows Server Primary VM (`WIN-J17IMHCEMA9`)	🔒 Active	❌ **NO** (Active Directory Domain Controller)
`192.168.1.11`	AdGuard Home Container (Port 53)	🔒 Active	❌ **NO** (Dedicated for AdGuard DNS)
`192.168.1.12 - 192.168.1.19`	Reserved for Future Static Servers (Web, DB, Linux)	🟢 **FREE**	☑ **YES** (Assign to new static server VMs)
`192.168.1.20`	`pro-win-client` (Lab Client VM 1)	🔒 Active	❌ **NO** (Client VM 1)
`192.168.1.21`	`pro-win-client2` (Lab Client VM 2)	🔒 Active	❌ **NO** (Client VM 2)
`192.168.1.22 - 192.168.1.49`	Reserved for Future Client Static VMs	🟢 **FREE**	☑ **YES** (Assign to client testing VMs)
`192.168.1.50`	`fileserver.e6.local` (Member File Server)	⚠ Reserved	⚠ Reserved for Storage / File Server
`192.168.1.100 - 192.168.1.254`	Windows Server DHCP Scope Pool	🔄 Dynamic	❌ **NO** (Dynamically leased by Windows DHCP)

The Core Networking Rule: 1 Virtual Machine = 1 IP Address

Virtual Machine in VMware	Its Dedicated IP
💻 Client VM 1 (pro-win-client)	192.168.1.20
💻 Client VM 2 (pro-win-client2)	192.168.1.21
🐧 New Linux / Web Server VM	192.168.1.12
🗄️ New Database Server VM	192.168.1.13
🗀 Storage / File Server VM	192.168.1.50

- **The Standard Rule:** In standard networking, **1 Virtual Machine = 1 unique IP Address**.
- **The ONLY Special Exception in Your Lab:** Your **Windows Server VM (`WIN-J17IMHCEMA9`)** holds **two IP addresses (`.10` and `.11`)** on the same virtual network card (`Ethernet0`).
- `192.168.1.10` ➔ Windows Server itself (Active Directory & Windows DNS)
- `192.168.1.11` ➔ AdGuard Docker container (Dedicated Port 53 socket)
- *Why?* Because we squeezed two DNS servers that both strictly required Port 53 onto that single machine without spinning up an extra VM.
- **For Every Other VM:** Always follow the standard rule: **1 VM = 1 unique IP address**. Never assign `.10` or `.11` to any other machine on the network!

Phase 0: VMware Workstation & Physical Host Pre-Configuration

To run Docker (WSL2) inside a Windows Server VM running on VMware Workstation, you are setting up **Nested Virtualization** ("a VM inside a VM").

Because modern Windows 11 hosts run **Virtualization-Based Security (VBS)** and **Hyper-V** by default, Windows locks the CPU's hardware virtualization extensions (VT-x). If you try to enable VT-x in VMware without unlocking the host first, VMware will fail to boot with:

`Feature 'hv.capable' was 0, but must be at least 0x1. Module 'FeatureCompatLate' power on failed.`

Follow these sub-phases carefully.

Phase 0.1: Unlock Hardware Virtualization on Physical Host (Laptop)

> ⚠ **WARNING —**
>
> **Host Linux / Docker Impact:**
>
> When you disable the host hypervisor lock, WSL2 (Ubuntu) and Docker Desktop on your **physical laptop** will temporarily be unable to start. They will resume working normally once you restore the settings (see **Phase 0.4** below).

1. **Turn Off Memory Integrity (Core Isolation):**

- On your physical host PC, open the Start Menu and search for **Core isolation** (or open **Windows Security ➔ Device Security ➔ Core isolation details**).
- Toggle **Memory integrity** to **OFF**.

2. **Disable Host Hyper-V Boot Lock:**

- Right-click the Start Menu on your physical PC and open **PowerShell as Administrator** (or Windows Terminal Admin).
- Run the following command:

```
bcdedit /set hypervisorlaunchtype off
```

- Ensure it returns: *"The operation completed successfully."*

3. **Boot into Windows 11.** VMware Workstation now has direct hardware access to Intel VT-x.

Phase 0.2: Configure VMware Workstation VM Settings

4. Ensure the Windows Server VM is **Powered Off**.
5. Right-click the VM ➔ Select **Settings**.
6. **Hardware ➔ Processors:**

- Check: ☑ **Virtualize Intel VT-x/EPT or AMD-V/RVI**.
- *(Do not check CPU performance counters or IOMMU unless specifically required).*

7. **Hardware ➔ Network Adapter (Choose Deployment Mode):**

- **Choice A: NAT Mode (`VMnet8`) [HIGHLY RECOMMENDED FOR LABS / LAPTOPS]:**
- Select **NAT: Used to share the host's IP address**.
- *(Benefits: Never breaks when switching between Home, School/Campus, or Cafe Wi-Fi. Bypasses 802.1X and captive portals).*
- **Choice B: Bridged Mode (`VMnet0`) [FOR PHYSICAL HOME DEVICES]:**
- Select **Bridged: Connected directly to the physical network**.
- Check **Replicate physical network connection state**.
- *Important:* In VMware main window, open **Edit ➔ Virtual Network Editor ➔ Change Settings (Admin) ➔** Select **VMnet0 ➔** Change **Bridged to:** from *Automatic* to your specific physical Wi-Fi card (e.g., `Intel(R) Wi-Fi 6E AX211 160MHz`).

8. Power on the Windows Server VM. It will now boot up cleanly with nested virtualization active.

Phase 0.3: Alternative - Run AdGuard Home Natively (No Host Changes Needed)

> **BEST PRACTICE & TIP —**
>
> If you do not want to disable Hyper-V on your physical laptop because you actively use Docker/Ubuntu on your host, you can run **AdGuard Home as a native Windows service** directly on Windows Server instead of inside Docker.
>
> 1. Download `AdGuardHome_windows_amd64.zip` inside the VM.
> 2. Extract to `C:\AdGuardHome`.
> 3. Run `.\AdGuardHome.exe -s install` in PowerShell.
>
> This requires **no nested virtualization (VT-x can stay OFF)** and uses only ~30 MB RAM while achieving the exact same ad-blocking and DoH results.

Phase 0.4: How to Revert & Restore Linux (WSL2 / Docker) on Your Physical Laptop

When you have finished your Windows Server lab and want your physical host's WSL2, Ubuntu, and Docker Desktop to work again:

9. **Power off the Windows Server VM** in VMware.
10. In VMware VM Settings ➔ **Processors** ➔ **UNCHECK** `Virtualize Intel VT-x/EPT or AMD-V/RVI`.

(If you leave this checked while the host hypervisor is re-enabled, VMware will show the `hv.capable` error again).

11. Open **PowerShell as Administrator** on your physical laptop and run:

```
bcdedit /set hypervisorlaunchtype auto
```

12. (Optional) In **Windows Security ➔ Device Security ➔ Core isolation details**, turn **Memory integrity** back to **ON**.
13. **Restart your physical laptop.**
14. Once rebooted, launch **Ubuntu (WSL2)** or **Docker Desktop** on your physical laptop. They will start normally!

Why we do this (Technical Rationale):

- **Why enable "Virtualize Intel VT-x/EPT"?**

AdGuard Home is packaged as a Linux-based container. Running Docker Desktop on Windows Server requires a Linux VM backend (WSL2 or Hyper-V). Because your Windows Server is *already* a virtual machine inside VMware, running Docker creates a "VM inside a VM" (**Nested Virtualization**). If you don't pass VT-x into VMware, Windows Server cannot run the WSL2 Linux kernel.

- **Why `bcdedit /set hypervisorlaunchtype off` is necessary on Windows 11?**

Windows 11 utilizes the Microsoft Hyper-V hypervisor for system security (VBS) and WSL2. Hyper-V monopolizes CPU VT-x instructions at Ring -1. Setting `hypervisorlaunchtype off` releases this lock so VMware Workstation can access VT-x directly.

- **Why Bridged Mode must be mapped manually?**

VMware's "Automatic" bridge detection often binds to secondary virtual adapters (such as VPN tunnels, Tailscale, or disconnected Ethernet ports) rather than the active Wi-Fi card. Manually pinning VMnet0 ensures stable layer-2 connectivity to your home/lab router.

Phase 1: Set Static IP on Windows Server

A DNS server must always maintain a fixed IP address. Choose the configuration matching your VMware network adapter setting from Phase 0.2.

Choice A: For NAT Mode (`VMnet8`) **[Recommended for Labs & Laptops]**

Use this mode if your VMware network adapter is set to **NAT**. It provides a fixed, reliable subnet that never breaks when moving between Home, School/Campus, or Cafe Wi-Fi.

1. Via PowerShell

```
# Get active network adapter interface alias (usually "Ethernet0")
Get-NetAdapter

# Set Static IP (192.168.1.10) and VMware NAT Gateway (192.168.1.1)
New-NetIPAddress -InterfaceAlias "Ethernet0" -IPAddress 192.168.1.10 -PrefixLength 24 -
DefaultGateway 192.168.1.1

# Set temporary public DNS (1.1.1.1 / 8.8.8.8) so VM has internet for downloading Docker
Set-DnsClientServerAddress -InterfaceAlias "Ethernet0" -ServerAddresses ("1.1.1.1",
"8.8.8.8")
```

2. Via GUI (`ncpa.cpl`)

15. Open **Network Connections** (`ncpa.cpl`).
16. Right-click your Ethernet adapter ➔ **Properties** ➔ Double-click **Internet Protocol Version 4 (TCP/IPv4)**.
17. Configure:

- **IP address:** `192.168.1.10`
- **Subnet mask:** `255.255.255.0`
- **Default gateway:** `192.168.1.1` (VMware Virtual NAT Gateway)

- **Preferred DNS server:** `1.1.1.1` *(switch to `127.0.0.1` after Phase 4)*
- **Alternate DNS server:** `8.8.8.8`

18. Click **OK → OK**.

Choice B: For Bridged Mode (`VMnet0`) **[For Fixed Home Wi-Fi]**

Use this mode ONLY if your VMware network adapter is set to **Bridged** and your laptop is on your fixed home Wi-Fi network.

1. Via PowerShell

```
# Set Static IP on home subnet and Home Router Gateway
New-NetIPAddress -InterfaceAlias "Ethernet0" -IPAddress 192.168.100.50 -PrefixLength 24 -
DefaultGateway 192.168.100.1

# Set temporary public DNS
Set-DnsClientServerAddress -InterfaceAlias "Ethernet0" -ServerAddresses ("1.1.1.1",
"8.8.8.8")
```

2. Via GUI (`ncpa.cpl`)

- **IP address:** `192.168.100.50`
- **Subnet mask:** `255.255.255.0`
- **Default gateway:** `192.168.100.1` (Home Router IP)
- **Preferred DNS server:** `1.1.1.1` *(switch to `127.0.0.1` after Phase 4)*
- **Alternate DNS server:** `8.8.8.8`

Why we do this (Technical Rationale):

- **Why a Static IP is Mandatory:**

If the server used dynamic DHCP, its IP address could change after reboot. Any client VM (`pro-win-client`) pointing to that DNS IP would immediately lose internet and domain name resolution.

- **Why use temporary public DNS (`1.1.1.1`) before switching to `127.0.0.1`?**

In Phase 1, the Windows DNS Server role and AdGuard Home container are not yet running. If you point DNS to `127.0.0.1` immediately, the server queries itself, receives no reply, and loses internet

access (blocking you from downloading Docker Desktop and WSL packages). Once Phase 4 is completed, DNS is switched to `127.0.0.1`.

Phase 2: Install Native Windows DNS Server Role

1. Execution Steps

Option A: Via PowerShell (Recommended)

```
Install-WindowsFeature -Name DNS -IncludeManagementTools
```

Option B: Via Server Manager GUI

19. Open **Server Manager ➔** Click **Add roles and features**.
20. Advance through wizard until **Server Roles**.
21. Check the box for **DNS Server** (Click **Add Features** when prompted).
22. Click **Next ➔ Next ➔ Install**.
23. Once complete, verify the service is running:

```
  Get-Service -Name DNS
```

💡 Why we do this (Technical Rationale):

- **Why Native Windows DNS instead of just using Docker?**

Native Windows DNS is fully integrated into the Windows kernel and Active Directory. It supports:

- **Dynamic DNS (DDNS):** Computers register their names automatically upon boot.
- **SRV & Kerberos Records:** Windows clients query `_ldap._tcp.dc._msdcs.<domain>` to find Domain Controllers for login authentication. Third-party DNS servers cannot manage these records reliably without breaking domain logins.

Phase 2.5: Install Docker on Windows Server

Because AdGuard Home is packaged as a Linux container, Docker needs a lightweight Linux engine to execute it.

1. Execution Steps (Docker Desktop with WSL2 Backend)

Step 1: Enable Virtual Machine Platform & Containers Features

You can enable these built-in features using either the Graphical User Interface (GUI) or PowerShell:

Option 1: Via Server Manager GUI (Visual Method)

24. Open **Server Manager** (from the Start Menu or Taskbar).
25. In the top-right corner, click **Manage ➔** select **Add Roles and Features**.
26. Click **Next** through:

- **Before You Begin ➔** Click **Next**
- **Installation Type ➔** Select *Role-based or feature-based installation* **➔** Click **Next**
- **Server Selection ➔** Select your local server **➔** Click **Next**

27. **Server Roles:** Click **Next** (no changes needed here).
28. **Features (Important):**

- Scroll down the list and check the box for **`Containers`**.
- When prompted, click **Add Features** to include management tools.

29. **Confirmation:**

- Check the box: **"Restart the destination server automatically if required"**.
- Click **Install**.

30. Once finished, restart the server.

Option 2: Via Classic Control Panel GUI (`appwiz.cpl`)

31. Press `Win + R` on your keyboard, type **`appwiz.cpl`**, and press **Enter**.
32. In the left sidebar, click **"Turn Windows features on or off"**.
33. In the feature tree, check:

- ☑ **Containers**
- ☑ **Virtual Machine Platform** (or **Hyper-V Platform**)

34. Click **OK**, let Windows apply changes, and click **Restart Now**.

Option 3: Via PowerShell (Fastest - One Command)

Open **PowerShell as Administrator**:

```
Enable-WindowsOptionalFeature -Online -FeatureName VirtualMachinePlatform -All -NoRestart
Enable-WindowsOptionalFeature -Online -FeatureName Containers -All -NoRestart

# Restart server to initialize hypervisor features
Restart-Computer
```

Step 2: Install WSL2 Linux Kernel

After the VM restarts, open PowerShell as Administrator:

```
wsl --install --no-distribution
wsl --update
```

Step 3: Download & Install Docker Desktop

Option A: Via Microsoft Edge Browser (Standard GUI Method)

35. Inside the Windows Server VM, open **Microsoft Edge**.
36. Navigate to: `https://www.docker.com/products/docker-desktop/`
37. Click the blue button: **Download for Windows** (downloads `Docker Desktop Installer.exe`, ~606 MB).
38. Once downloaded, open your **Downloads** folder and double-click `Docker Desktop Installer.exe`.

Option B: Via PowerShell (Automated Download)

Open PowerShell as Administrator:

```
Invoke-WebRequest -Uri
"https://desktop.docker.com/win/main/amd64/Docker%20Desktop%20Installer.exe" -OutFile
"$env:TEMP\DockerDesktopInstaller.exe"
Start-Process "$env:TEMP\DockerDesktopInstaller.exe" -Wait
```

Installer Configuration Screen (What to choose):

When the **"Configuration"** window appears:

- **Option 1 (Default & Recommended):** Keep **`Per-user installation (Recommended)`** selected. This automatically uses the **WSL 2 backend**.

- **Option 2 (All Users):** If you select **`All-users installation`**, make sure the checkbox **"Use WSL 2 instead of Hyper-V"** is **CHECKED** (do NOT check "Allow Windows Containers").
- Keep **"Add shortcut to desktop"** checked.
- Click **OK**.

Post-Installation:

39. Wait 2–3 minutes while packages unpack and install.
40. When the installation completes, click **Close and restart** (or sign out and log back in).
41. Once back on your desktop, double-click the **Docker Desktop** shortcut.
42. Accept the Docker Service Agreement. Docker Desktop will start up with its green engine indicator!

Step 4: Verify Docker Engine

Open PowerShell and verify:

```
docker --version
docker compose version
docker info
```

Why we do this (Technical Rationale):

- **Why Docker for AdGuard Home?**

Running AdGuard in Docker isolates it from the Windows host OS. If you want to update AdGuard, you simply pull the new image—no registry edits, no system file corruption, and no complicated Linux installation.

- **Why WSL2 instead of Hyper-V containers?**

WSL2 provides a real, optimized Linux kernel with direct memory management and near-native speed, consuming fewer system resources (RAM/CPU) than running a full secondary virtual machine.

Phase 3: Deploy AdGuard Home in Docker

1. Execution Steps

Step 1: Create Project Directory

```
mkdir C:\adguard
cd C:\adguard
```

Step 2: Create `docker-compose.yml`

Create `C:\adguard\docker-compose.yml`:

```
services:
  adguardhome:
    image: adguard/adguardhome:latest
    container_name: adguardhome
    restart: unless-stopped
    ports:
      # Bind directly to dedicated secondary IP on port 53 (Avoids Windows DNS conflict)
      - "192.168.1.11:53:53/tcp"
      - "192.168.1.11:53:53/udp"
      # Initial setup wizard port
      - "3000:3000/tcp"
      # Web Admin dashboard
      - "8080:80/tcp"
    volumes:
      - ./workdir:/opt/adguardhome/work
      - ./confdir:/opt/adguardhome/conf
```

Step 3: Launch the Container

```
cd C:\adguard
docker compose up -d
```

Verify status:

```
docker ps
```

Why we do this (Technical Rationale):

- **Why bind AdGuard to secondary IP `192.168.1.11:53`?**

Native Windows DNS is already listening on `192.168.1.10:53`. Windows DNS Forwarders only know how to forward to port `53` (they cannot forward to custom ports like `:5353`). Binding AdGuard to `192.168.1.11:53` gives AdGuard its own dedicated port 53 socket on the same network card without any conflict.

- **Why use persistent volumes (`./workdir` and `./confdir`)?**

Containers are ephemeral by design (if a container restarts or updates, everything inside is wiped). Volume mappings store AdGuard's blocklists, query logs, and admin passwords in folders on the Windows Server hard drive (`C:\adguard\confdir`), ensuring data survives updates.

- **Why `restart: unless-stopped`?**

Ensures AdGuard automatically boots up when Windows Server starts or after a power outage.

Phase 4: Configure AdGuard Home & Cloudflare DoH

1. Execution Steps

Step 1: Complete Initial Setup Wizard

43. Open browser: `http://localhost:3000` (or `http://192.168.1.10:3000`).
44. Click **Get Started**.
45. **Admin Web Interface:** Set listen port to `80` (mapped to `8080` outside).
46. **DNS Server:** Set listen port to `53` (mapped to `192.168.1.11:53` outside).
47. Set **Admin Username** and **Password** ➔ Click **Next** ➔ **Finish**.

Step 2: Configure Encrypted Upstream DNS

48. Open AdGuard dashboard at `http://localhost:8080` (or `http://192.168.1.10:8080`) and log in.
49. Go to **Settings** ➔ **DNS Settings**.
50. Under **Upstream DNS servers**, clear default entries and paste:

```
   # Cloudflare DNS-over-HTTPS via Direct IP (Bypasses UDP 53 blocking and domain
bootstrap)
   https://1.1.1.1/dns-query
```

```
# Google DNS-over-HTTPS via Direct IP
https://8.8.8.8/dns-query

# Alternatively: DNS-over-TLS or TCP DNS
# tls://1.1.1.1
# tcp://1.1.1.1
```

51. **Bootstrap DNS servers (Crucial Setting):**

- Scroll down the page to "**Bootstrap DNS servers**".
- Replace the defaults with:

```
1.1.1.1
8.8.8.8
192.168.1.1
```

- *Why?* If you use a domain name like `dns.cloudflare.com`, AdGuard must resolve the domain before connecting. If UDP 53 is blocked by your ISP or Docker NAT, bootstrapping fails. Using direct IP `https://1.1.1.1/dns-query` avoids this issue completely!

52. Scroll down and ensure **DHCP is disabled** (AdGuard DHCP is OFF by default).
53. Click **Apply** and verify with **Test upstreams** (it will now show green checkmarks!).

Why we do this (Technical Rationale):

- **Why Cloudflare DoH (`https://dns.cloudflare.com/dns-query`)?**

Standard DNS sends domain queries in plain text over UDP 53. ISPs can inspect and log every domain you visit. By using DNS-over-HTTPS, queries are encrypted with TLS 1.3 over TCP port 443. The ISP only sees encrypted traffic to Cloudflare's IP (`1.1.1.1`), keeping your browsing private.

- **Why Google (`8.8.8.8`) as secondary fallback?**

If Cloudflare experiences an outage or fiber routing issue, AdGuard automatically falls back to Google DNS, ensuring high availability.

- **Why keep AdGuard DHCP disabled?**

Having two DHCP servers on the same network causes a **DHCP Race Condition** where devices receive conflicting network configurations.

Phase 5: Connect Windows DNS to AdGuard (Forwarder)

1. Execution Steps

Step 1: Restrict Windows DNS to Primary IP (`192.168.1.10`)

By default, Windows DNS listens on `0.0.0.0:53` (all interfaces), blocking Docker from binding port 53. Restrict Windows DNS to only listen on your VM's primary static IP (`192.168.1.10`):

```
# Restrict Windows DNS to only listen on primary adapter IP
dnscmd . /resetlistenaddresses 192.168.1.10

# Restart Windows DNS service
Restart-Service DNS
```

Step 2: Assign Secondary Dedicated IP (`192.168.1.11`) for AdGuard

Add a secondary IP to `Ethernet0` so AdGuard has its own dedicated endpoint on port 53.

Option A: Via PowerShell

```
New-NetIPAddress -InterfaceAlias "Ethernet0" -IPAddress 192.168.1.11 -PrefixLength 24
```

Option B: Via Windows GUI (`ncpa.cpl`)

54. Press `Win + R`, type **`ncpa.cpl`**, and press **Enter**.
55. Right-click **Ethernet0 ➔ Properties**.
56. Double-click **Internet Protocol Version 4 (TCP/IPv4) ➔** Click **Advanced...**.
57. Under **IP addresses**, click **Add...**.

- **IP address:** `192.168.1.11`
- **Subnet mask:** `255.255.255.0`

58. Click **Add ➔ OK ➔ OK ➔ Close**.

Step 3: Bind AdGuard to `192.168.1.11:53` in `docker-compose.yml`

In `C:\adguard\docker-compose.yml`, bind AdGuard to the secondary IP on standard port 53:

```
services:
  adguardhome:
    image: adguard/adguardhome:latest
```

```
    container_name: adguardhome
    restart: unless-stopped
    ports:
      - "192.168.1.11:53:53/tcp"
      - "192.168.1.11:53:53/udp"
      - "8080:80/tcp"
    volumes:
      - ./workdir:/opt/adguardhome/work
      - ./confdir:/opt/adguardhome/conf
```

Apply the changes:

```
cd C:\adguard
docker compose up -d
```

Step 4: Configure AdGuard Conditional Forwarding to Windows DNS

> **NOTE —**
>
> **Why Windows DNS cannot forward to AdGuard (Error 9552):**
>
> Running `Set-DnsServerForwarder -IPAddress 192.168.1.11` fails with `WIN32 9552 (DNS_ERROR_CANNOT_FORWARD_TO_SELF)` because Windows DNS refuses to forward to an IP that belongs to the local machine.
>
> **The Production Solution:** Let AdGuard handle the routing! AdGuard has no 9552 restriction.

59. Open AdGuard dashboard: `http://localhost:8080/#dns`
60. Under **Upstream DNS servers**, paste the complete configuration:

```
   # Internal domain zones (with dots: FQDN)
   [/e6.local/lab.internal/]192.168.1.10:53

   # Single-label local machine names (without dots: e.g., ping fileserver)
   [//]192.168.1.10:53

   # Public Internet (Encrypted DNS-over-HTTPS via Direct IP)
   https://1.1.1.1/dns-query
   https://8.8.8.8/dns-query
```

61. Click **Apply**.

🔍 Deep Dive: Understanding `[//]` vs. `[/e6.local/lab.internal/]` (Dots vs. No Dots)

A foundational concept in AdGuard upstream routing is understanding how it differentiates between **Full Domain Names (FQDN)** and **Short Computer Names (Single-Label Hostnames)**:

- **Domain Zones ALWAYS have dots:** `app.lab.internal` (2 dots), `WIN-J17IMHCEMA9.e6.local` (2 dots), `google.com` (1 dot).
- **Short Computer Names have ZERO dots:** `app` (0 dots), `fileserver` (0 dots), `WIN-J17IMHCEMA9` (0 dots).

1. What `[//]192.168.1.10:53` Actually Does

Notice there is **nothing between the slashes** (`//`). In AdGuard / dnsmasq syntax, an empty domain matches:

> **"Match ONLY names with ZERO dots"** *(Single-label hostnames)*

- `ping fileserver` ➔ **0 dots** ➔ Matches `[//]` ☑ (Forwarded to Windows DNS `.10`)
- `ping app` ➔ **0 dots** ➔ Matches `[//]` ☑ (Forwarded to Windows DNS `.10`)
- `app.lab.internal` ➔ **Has dots! ➔ IGNORED by `[//]`** ❌

2. What `[/e6.local/lab.internal/]` Actually Does

Notice the names **are written between the slashes**. This tells AdGuard:

> **"Match any query ending with `.e6.local` OR `.lab.internal`"**

- `app.lab.internal` ➔ Ends with `.lab.internal` ➔ Matches ☑ (Forwarded to Windows DNS `.10`)
- `WIN-J17IMHCEMA9.e6.local` ➔ Ends with `.e6.local` ➔ Matches ☑ (Forwarded to Windows DNS `.10`)
- `google.com` ➔ Doesn't match ➔ Forwarded to Cloudflare DoH (`1.1.1.1`)

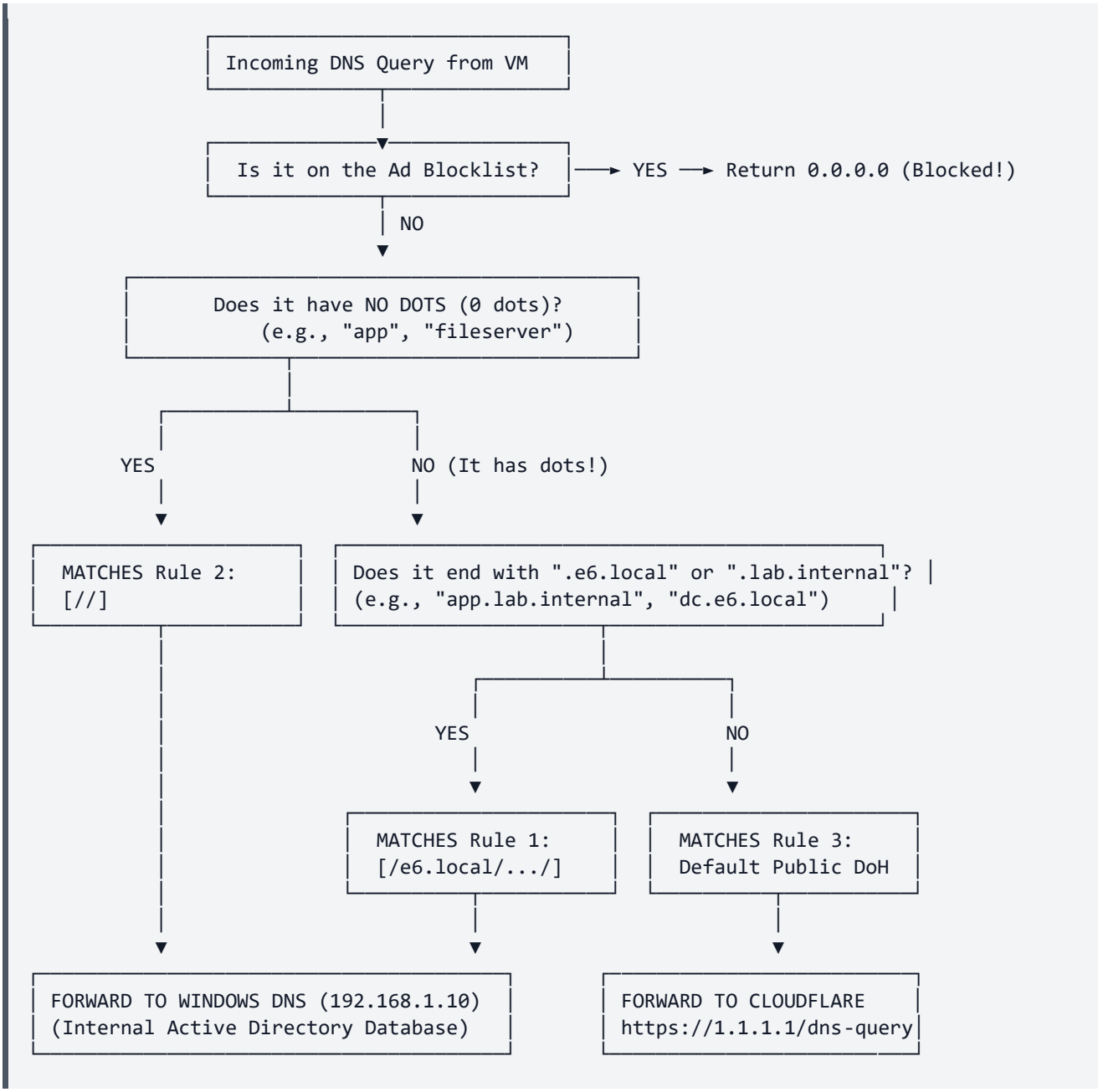
3. Why `[//]` Does NOT Mean "All Zones" (The # Comment Hazard!)

If you comment out `# [/e6.local/lab.internal/]` thinking `[//]` catches everything:

- When a device queries `app.lab.internal`, AdGuard sees dots.
- It skips `[//]`.

- It falls back to Cloudflare (`1.1.1.1`), which responds with `NXDOMAIN` (Does not exist!).
- **Result:** Internal domain resolution breaks. You **MUST** keep both rules active.

4. How Both Work Together (Decision Flowchart)



5. Evaluation Matrix

What you type	Dots	Matches `[//]`?	Matches `[/e6.local/lab.internal/]`?	Destination Resolved

What you type	Dots	Matches `[//]`?	Matches `[/e6.local/lab.internal/]`?	Destination Resolved
`nslookup app`	**0**	☑ **YES**	✗ No	Windows DNS (`192.168.1.10`)
`nslookup fileserver`	**0**	☑ **YES**	✗ No	Windows DNS (`192.168.1.10`)
`nslookup app.lab.internal`	**2**	✗ **NO**	☑ **YES**	Windows DNS (`192.168.1.10`)
`nslookup WIN-J17IMHCEMA9.e6.local`	**2**	✗ **NO**	☑ **YES**	Windows DNS (`192.168.1.10`)
`nslookup google.com`	**1**	✗ **NO**	✗ No	Cloudflare DoH (`1.1.1.1:443`)

6. Step-by-Step Resolution Scenarios

- **Scenario A: Full Domain Lookup (`nslookup app.lab.internal`):**

AdGuard checks `[//]` (skipped: contains dots) ➔ matches `[/lab.internal/]` ➔ forwards to Windows DNS `.10` ➔ returns `192.168.1.77`.

- **Scenario B: Short Machine Name (`nslookup app` or `ping fileserver`):**

AdGuard checks `[//]` ➔ matches (zero dots) ➔ forwards to Windows DNS `.10` ➔ Windows DNS appends search suffix and returns `192.168.1.77`.

- **Scenario C: Public Web Request (`nslookup google.com`):**

AdGuard checks `[//]` (no) ➔ checks local zones (no) ➔ encapsulates in TLS 1.3 over TCP 443 to Cloudflare DoH (`https://1.1.1.1/dns-query`) ➔ returns public IP in ~25ms.

Step 5: Point Network Adapters & Client VMs to AdGuard (`192.168.1.11`)

Now point Windows Server's own network adapter (and all client VMs like `pro-win-client`) to AdGuard:

Option A: Via PowerShell (Automated)

```
Set-DnsClientServerAddress -InterfaceAlias "Ethernet0" -ServerAddresses ("192.168.1.11")
```

Option B: Via Windows Graphical User Interface (GUI)

62. **Open Network Connections GUI:**

- Press `Win + R` on your keyboard.
- Type **`ncpa.cpl`** and press **Enter**.
- Right-click **Ethernet0** ➔ select **Properties**.

63. **Set IPv4 DNS to AdGuard (`192.168.1.11`):**

- Double-click **Internet Protocol Version 4 (TCP/IPv4)**.
- In the bottom section, select: "**Use the following DNS server addresses**":
- **Preferred DNS server:** `192.168.1.11`
- **Alternate DNS server:** *(leave completely blank / empty)*
- Click **OK**.

64. **Clear IPv6 `::1` (Crucial: Prevents Windows from Bypassing AdGuard!):**

- In that same *Ethernet0 Properties* window, double-click **Internet Protocol Version 6 (TCP/IPv6)**.
- Make sure it is set to "**Obtain DNS server address automatically**" (or uncheck the IPv6 checkbox entirely).
- Click **OK**, then click **Close**.

🩹 Step 5.1: Test in PowerShell (Notice: No IP Needed!):

```
# 1. Tests through AdGuard -> Returns 0.0.0.0 (Ad Blocked!)
nslookup adservice.google.com

# 2. Tests through AdGuard -> Cloudflare DoH (Encrypted Internet!)
nslookup google.com

# 3. Tests through AdGuard -> Windows DNS .10 (Internal Domain!)
nslookup WIN-J17IMHCEMA9.e6.local
```

(Notice: `Address: 192.168.1.11` answers automatically as your default DNS resolver!)

💡 Why we do this (Technical Rationale):

- **Why assign a secondary IP (`192.168.1.11`)?**

Windows Server DNS (`dns.exe`) retains a low-level lock on loopback (`127.0.0.1:53`) for internal Windows security services (Kerberos/Active Directory). By assigning a secondary IP (`192.168.1.11`)

to `Ethernet0` and restricting Windows DNS to `192.168.1.10`, port `53` on `192.168.1.11` is 100% available for Docker.

- **Why does AdGuard forward to Windows DNS (`[/e6.local/]192.168.1.10:53`)?**

This elegantly bypasses Microsoft's hardcoded Error 9552. AdGuard receives all incoming queries on standard port 53. If the query ends in `.e6.local`, AdGuard forwards it directly to Windows DNS on `.10`. If the query is an ad or tracker, AdGuard blocks it (`0.0.0.0`). If it is for the internet, AdGuard encrypts it to Cloudflare over port 443.

- **Why point Ethernet0 DNS to `192.168.1.11`?**

This ensures the Windows Server itself, background applications, and all client VMs enjoy network-wide ad blocking, encrypted upstream privacy, and instant resolution of internal Active Directory records.

Phase 6: Local Authoritative Zone & Host Records (Testing)

> 📌 **NOTE —**
>
> **Active Directory Domain Controller Note:**
>
> If this server is already promoted to a Domain Controller (as shown by `e6.local` existing as an **Active Directory-Integrated Primary Zone**), **do NOT create a new zone!** Windows Server already created it automatically. Click **Cancel** on the New Zone Wizard and proceed directly to **Step 2 (Add Test Record)**.

1. Execution Steps

Step 1: Check Existing Zone (or Create New if Standalone DNS)

- **If `e6.local` already exists (AD DC):** Simply click on `e6.local` in the left tree.
- **If on a standalone DNS server (No AD):**

65. In **DNS Manager**, expand server name.
66. Right-click **Forward Lookup Zones** ➔ Select **New Zone...**.
67. Select **Primary zone** ➔ Click **Next**.
68. Zone Name: Type `e6.local` ➔ Click **Next** ➔ **Finish**.

Step 2: Add Test Host Record (`fileserver`)

69. Click on the existing **`e6.local`** folder in the left pane.
70. In the right pane (or right-click `e6.local`), select **New Host (A or AAAA)...**.
71. Configure the test record:

- **Name:** `fileserver`
- **IP address:** `192.168.1.50` (or your intended file server IP)
- Check **Create associated pointer (PTR) record** (optional).

72. Click **Add Host** ➔ Click **OK** ➔ Click **Done**.

💡 Why we do this (Technical Rationale):

- **Why does Active Directory create this zone automatically?**

Active Directory relies on DNS for service discovery. When you promote a server to a Domain Controller, Windows automatically builds the AD-integrated zone with SRV records (`_msdcs`), allowing domain computers to locate login authenticators (Kerberos/LDAP).

- **Why add an authoritative test record?**

Adding `fileserver.e6.local` demonstrates that any query matching your internal namespace is answered immediately from Windows DNS without reaching AdGuard or the public internet.

Phase 7: Verification & Testing Suite

Run these tests in PowerShell on the **Windows Server VM** (or from client VM `pro-win-client`). Because DNS was set to `192.168.1.11` in Step 5, you no longer need to type an IP address!

Test 1: Verify Active Directory & Local Domain Resolution (AdGuard ➔ Windows DNS)

```
# Queries default DNS (192.168.1.11) -> AdGuard forwards [/e6.local/] to Windows DNS
(192.168.1.10:53)
nslookup WIN-J17IMHCEMA9.e6.local
```

- **Expected Result:** Returns addresses `192.168.1.10` and `192.168.1.11`.
- **Rationale:** Proves AdGuard's conditional rule `[/e6.local/]192.168.1.10:53` correctly routes domain queries to Windows DNS, preserving full Active Directory integration without any loops.

Test 2: Verify Internet Resolution (Cloudflare DoH via AdGuard)

```
# Queries default DNS (192.168.1.11) -> AdGuard resolves via Cloudflare DoH
nslookup google.com
```

- **Expected Result:**

```
Server:  UnKnown
Address:  192.168.1.11

Non-authoritative answer:
Name:    google.com
Addresses: 142.250.4.139, 142.250.4.100, ...
```

- **Rationale:** Proves clean public internet queries are encrypted over TLS 1.3 / Port 443 via Cloudflare DoH (`https://1.1.1.1/dns-query`) in ~25ms.

Test 3: Verify Ad-Blocking & Threat Sinkhole

```
# Queries default DNS (192.168.1.11) -> AdGuard blocks advertising / tracking domain
nslookup adservice.google.com
```

- **Expected Result:**

```
Server:  UnKnown
Address:  192.168.1.11

Non-authoritative answer:
Name:    adservice.google.com.e6.local
Addresses:  ::
          0.0.0.0
```

- **Rationale:** Proves AdGuard intercepts known ad/telemetry domains at the DNS boundary and returns `0.0.0.0` before any web traffic can leave the machine.

Test 4: Inspect AdGuard Dashboard Query Log

73. Open `http://localhost:8080` (or `http://192.168.1.10:8080`) and click **Query Log**.
74. Notice `google.com` is marked as **Processed** (encrypted upstream: `https://1.1.1.1/dns-query`).

75. Notice `adservice.google.com` is highlighted in **RED as Blocked** (Rule: AdGuard DNS filter).

Phase 7.2: Client VM Setup & End-to-End Verification (`pro-win-client`)

This section guides configuring any separate client virtual machine (such as **`pro-win-client`**) to utilize the hybrid DNS infrastructure.

1. Architectural Prerequisites

- **Same Network Segment:** Both `pro-win-server` and `pro-win-client` must be attached to the same VMware virtual adapter: **`VMnet8 (NAT)`**.
- **Domain Membership Not Required:** The client VM does **NOT** need to join the Active Directory domain to use this DNS server. Any workgroup PC, Linux client, or mobile device on the subnet can use it.

2. Windows Server Firewall Configuration (Allow Port 53 Inbound)

By default, Windows Server Firewall blocks inbound UDP Port 53 packets directed to third-party services like Docker.

Option A: Via GUI (`wf.msc`)

76. On the **Windows Server VM**, press `Win + R` ➔ type **`wf.msc`** ➔ press **Enter**.
77. Click **Inbound Rules** ➔ click **New Rule...** (right panel).
78. **Rule Type:** Select **Port** ➔ click **Next**.
79. **Protocol and Ports:** Select **UDP**, type specific local port **`53`** ➔ click **Next**.
80. **Action:** Select **Allow the connection** ➔ click **Next**.
81. **Profile:** Ensure **Domain**, **Private**, and **Public** are checked ➔ click **Next**.
82. **Name:** Type **`DNS Port 53 (UDP)`** ➔ click **Finish**.
83. *(Repeat same steps for TCP port 53).*

Option B: Via PowerShell (One-Liner on Windows Server)

```
New-NetFirewallRule -DisplayName "Allow DNS Inbound (UDP 53)" -Direction Inbound -
LocalPort 53 -Protocol UDP -Action Allow
New-NetFirewallRule -DisplayName "Allow DNS Inbound (TCP 53)" -Direction Inbound -
LocalPort 53 -Protocol TCP -Action Allow
```

3. Configure Client VM Adapter (`ncpa.cpl`)

On the **`pro-win-client`** machine:

Option A: Via GUI (`ncpa.cpl`)

84. Press `Win + R` ➔ type **`ncpa.cpl`** ➔ press **Enter**.
85. Right-click **Ethernet0** ➔ select **Properties**.
86. Double-click **Internet Protocol Version 4 (TCP/IPv4)**:

- **IP address:** `192.168.1.20`
- **Subnet mask:** `255.255.255.0`
- **Default gateway:** `192.168.1.1`
- **Preferred DNS server:** `192.168.1.11` *(AdGuard Home)*
- **Alternate DNS server:** *(leave completely blank)*
- Click **OK**.

87. Double-click **Internet Protocol Version 6 (TCP/IPv6)**:

- Set to **Obtain DNS server address automatically** (or uncheck IPv6).
- Click **OK** ➔ click **Close**.

Option B: Via PowerShell

```
Set-DnsClientServerAddress -InterfaceAlias "Ethernet0" -ServerAddresses ("192.168.1.11")
```

4. Run Client Verification Suite

In PowerShell on the **client machine (`pro-win-client`)**:

```
# 1. Test Layer 3 Connectivity (Ping to Server & AdGuard IP)
ping 192.168.1.10
ping 192.168.1.11

# 2. Test Active Directory Discovery (Resolves Domain Controller IPs)
nslookup WIN-J17IMHCEMA9.e6.local

# 3. Test Network-Wide Ad Blocking (Returns 0.0.0.0 Sinkhole)
nslookup adservice.google.com

# 4. Test Encrypted Public Internet Resolution (Cloudflare DoH)
nslookup google.com
```

5. Verify Live Queries in AdGuard Dashboard

88. Open the AdGuard dashboard: `http://192.168.1.10:8080` (or `http://localhost:8080`).
89. Navigate to **Query Log**.
90. Confirm that queries originating from client IP **`192.168.1.20`** appear in real-time, categorized into **Processed** (green) and **Blocked** (red).

Phase 8: Troubleshooting & Diagnostic Reference

1. Diagnose Port 53 Listeners & Ownership

If Docker throws a port collision error, inspect exactly which process owns port 53 across all IP addresses:

```
# Check UDP Port 53 listeners
Get-NetUDPEndpoint -LocalPort 53 | Format-Table LocalAddress, LocalPort, OwningProcess

# Check TCP Port 53 listeners
Get-NetTCPConnection -LocalPort 53 | Format-Table LocalAddress, LocalPort, OwningProcess,
State

# Identify the process name by PID
Get-Process -Id (Get-NetUDPEndpoint -LocalPort 53).OwningProcess -ErrorAction
SilentlyContinue
```

- **Expected State:**
- `192.168.1.10:53` ➔ Owned by `dns.exe` (Windows DNS).
- `127.0.0.1:53` ➔ Locked by `dns.exe` (Windows internal authentication).
- `192.168.1.11:53` ➔ Owned by `com.docker.backend.exe` (AdGuard Home).

2. Resolving Docker Bind Errors

Error A: `bind: Only one usage of each socket address is normally permitted (127.0.0.1:53)`

- **Root Cause:** Microsoft Windows DNS (`dns.exe`) retains a kernel-level lock on loopback `127.0.0.1:53` for Active Directory and Kerberos ticket issuance. It cannot be unbound.
- **Solution:** Do not bind to `127.0.0.1`. Use the secondary IP `192.168.1.11:53` in `docker-compose.yml`.

Error B: `listen udp4 192.168.1.11:53: can't bind on the specified endpoint`

- **Root Cause:** The IP `192.168.1.11` does not exist on `Ethernet0`, or is still in `Tentative` state (ARP duplicate address detection).
- **Solution:** Run `New-NetIPAddress -InterfaceAlias "Ethernet0" -IPAddress 192.168.1.11 -PrefixLength 24`. Verify with `ipconfig /all` that `192.168.1.11` shows `(Preferred)` before running `docker compose up -d`.

---

3. Inspect AdGuard Container Logs

If upstream servers show red error banners:

```
docker logs --tail 40 adguardhome
```

- **Common Log Insights:**
- `x509: certificate expired / invalid` ➔ System clock drifted. Run `w32tm /resync /force`.
- `read: connection refused / timeout` ➔ Outbound UDP 53 blocked. Switch upstream to direct IP DoH: `https://1.1.1.1/dns-query`.
- `bootstrap DNS timeout` ➔ Clear default IPv6 addresses (`2620:fe::10`) in AdGuard Bootstrap DNS settings and enter `1.1.1.1`, `8.8.8.8`, `192.168.1.1`.

---

4. Direct Tier-by-Tier Testing

```
# Test Tier 2 (AdGuard Docker) directly on standard port 53:
nslookup google.com 192.168.1.11

# Test Tier 1 (Windows DNS Server) on standard port 53:
nslookup google.com 192.168.1.10

# Test Local Zone Authority (Windows DNS):
nslookup fileserver.e6.local 192.168.1.10

# Test Ad Sinkhole (AdGuard):
nslookup adservice.google.com 192.168.1.10
```

5. Open Windows Firewall for Lab Clients

If client VMs (`pro-win-client`) cannot reach Windows DNS or AdGuard:

```
New-NetFirewallRule -DisplayName "Inbound DNS (UDP 53)" -Direction Inbound -LocalPort 53
-Protocol UDP -Action Allow
New-NetFirewallRule -DisplayName "Inbound DNS (TCP 53)" -Direction Inbound -LocalPort 53
-Protocol TCP -Action Allow
New-NetFirewallRule -DisplayName "AdGuard Web UI (TCP 8080)" -Direction Inbound -
LocalPort 8080 -Protocol TCP -Action Allow
```